프론트엔드 데이터 처리

01 프론트엔드 데이터 처리 개념

02 프론트엔드 보안 이슈

03 프론트엔드 보안 개발 방법

📌 학습 목표

- ✅ 웹 애플리케이션에서 프론트엔드 데이터 처리가 보안에 미치는 영향을 이해한다.
- ✅ 클라이언트 측(JavaScript)에서 발생할 수 있는 보안 취약점을 학습한다.
- ✅ 보안 강화를 위한 프론트엔드 개발 및 데이터 처리 방법을 익힌다.

📍 학습 내용

1. 프론트엔드 데이터 처리 개념
2. 프론트엔드 보안 이슈
3. 프론트엔드 보안 개발 방법

프론트엔드 데이터 처리

01 프론트엔드 데이터 처리 개념

02 프론트엔드 보안 이슈

03 프론트엔드 보안 개발 방법

클라이언트/서버 구조

정의 및 기본 개념

· 클라이언트: 사용자 인터페이스(UI)를 제공하며, 사용자의 요청(예: 브라우저, 모바일 앱)이 발생하는 프론트엔드 부분

· 서버: 클라이언트의 요청을 처리하고, 필요한 데이터를 생성하거나 제공하는 백엔드 시스템

통신 방식: 웹 환경에서 클라이언트와 서버는 주로 HTTP/HTTPS 프로토콜을 통해 네트워크 상에서 데이터를 주고받음

구조적 특징

· 분리된 역할: 클라이언트는 사용자 경험에 집중, 서버는 데이터 처리와 비즈니스 로직 수행

· 중앙 집중식 처리: 서버가 모든 요청을 받아 처리하고, 클라이언트는 결과를 화면에 표시

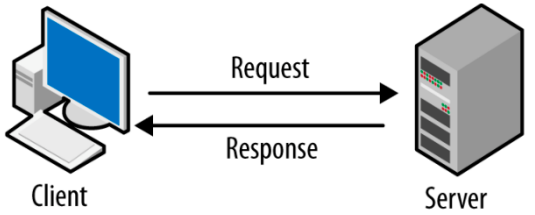
· 확장성: 서버를 확장하여 여러 클라이언트의 요청을 처리할 수 있음

통신 흐름에 따른 처리 형태

① 클라이언트 : [요청 전송] 사용자가 서버로 웹 페이지를 요청하거나 데이터를 입력

② 서버 : [요청 처리] 서버가 요청을 받아 비즈니스 로직 실행, 데이터베이스 연동 등 수행

③ 서버 : [응답 전송] 처리 결과를 클라이언트에 반환, 웹 페이지 렌더링 또는 API 응답 제공



프론트엔드의 역할

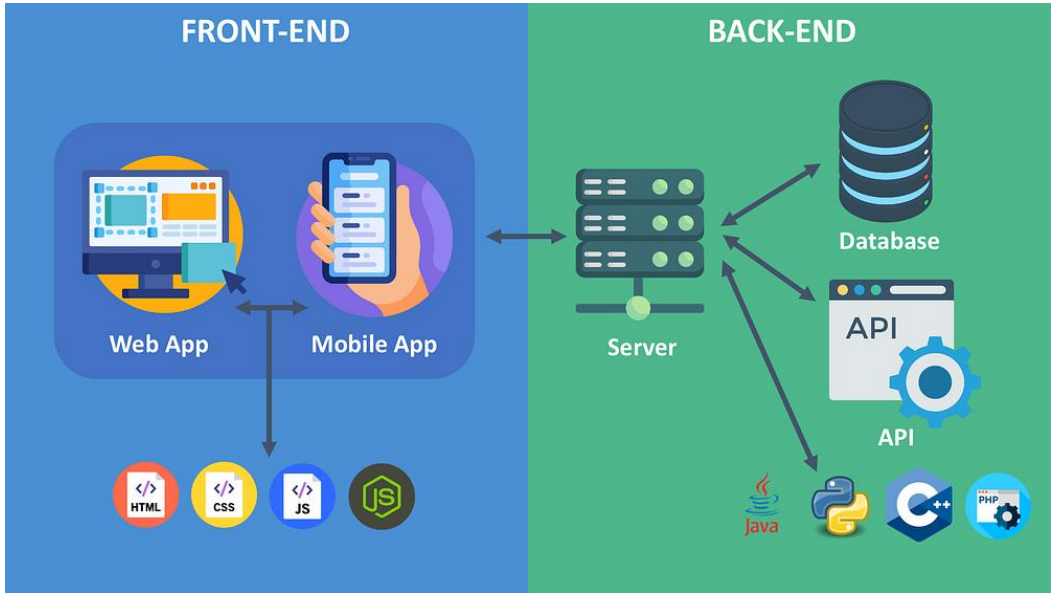
프론트엔드 정의

사용자와 직접 상호작용하는 웹 애플리케이션의 "보이는 부분"

브라우저에서 실행되며, HTML, CSS, JavaScript 등을 활용

주요 역할

- UI/UX 제공
  - 사용자 친화적인 인터페이스 설계 및 반응형 디자인 구현
  - 사용자 경험(UX) 개선 및 접근성 고려
- 데이터 시각화 및 인터랙션
  - 사용자 입력 처리, 애니메이션 및 동적 콘텐츠 제공
  - 클라이언트-서버 간 데이터 교환 및 API 호출 처리
- 표출 성능 최적화
  - 페이지 로딩 속도 향상, 리소스 압축 및 캐싱 전략 적용



출처: https://medium.com/@adityagaba1322/streamlining-backend-frontend-integration-a-quick-guide-145eca3cca05

프론트엔드에서 입력 데이터 처리 방식

입력 데이터 캡처

- 이벤트 리스너(Event Listener) 활용

  사용자의 입력 이벤트(예: `onChange`, `onInput`, `onSubmit`)를 통해 입력 데이터를 캡처

- 양방향 데이터 바인딩

  React, Angular, Vue 등 프레임워크에서 제공하는 바인딩 기법으로 UI와 상태를 동기화

입력 데이터 유효성 검증

- HTML5 내장 검증

  `required`, `pattern`, `min`, `max` 등의 속성으로 기본 검증 수행

- 클라이언트 사이드 JavaScript 검증

  정규표현식 및 커스텀 함수로 복잡한 유효성 검증 로직 적용

- 즉각적 피드백 제공

  실시간 에러 메시지 표시 및 UI 업데이트를 통해 사용자 경험 향상

프론트엔드에서 입력 데이터 처리 방식

입력 데이터 보안 처리

- 입력값 이스케이핑(Escaping) 및 필터링

  XSS 및 인젝션 공격 방지를 위해 사용자 입력값을 안전하게 이스케이프 처리

  서드파티 라이브러리(예: DOMPurify) 사용하여 HTML 및 스크립트 태그 제거

- 정적 분석 및 테스트

  클라이언트 코드 내 입력 처리 로직에 대한 보안 리뷰 및 단위 테스트 수행

입력 데이터 서버로 전송

- 안전한 AJAX/Fetch 요청

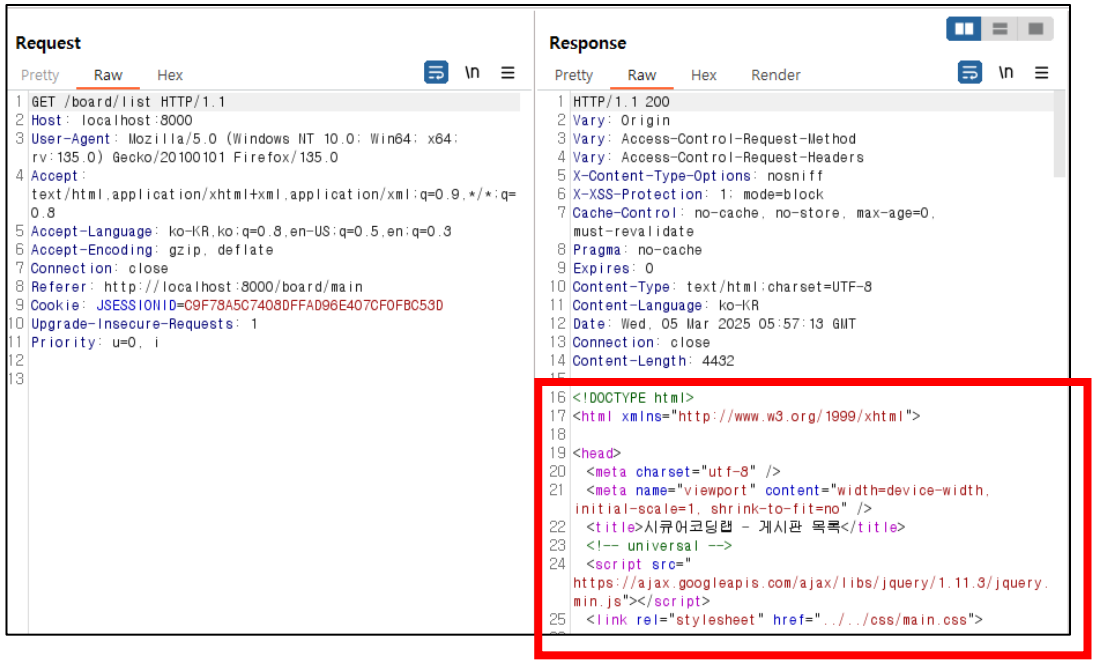
  HTTPS를 통한 암호화된 통신으로 데이터 전송

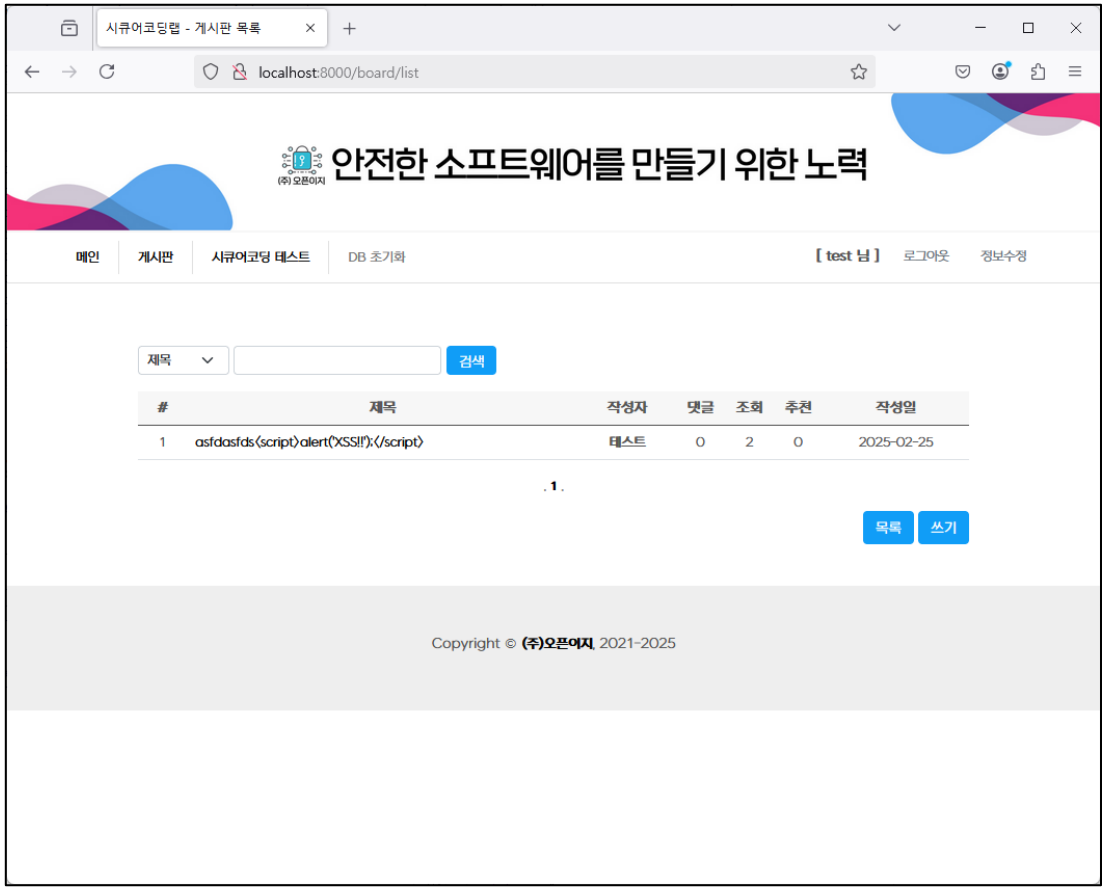
- 오류 처리 및 로깅

  전송 실패나 검증 오류 시 사용자에게 명확한 메시지 제공 및 클라이언트 측 로그 기록

프론트엔드에서 입력 데이터 처리 방식

1. 특정 페이지 요청을 백엔드가 받아 적절히 HTML을 완성하여 응답한 것을 렌더링하여 화면 표출
   · Static HTML(화면구성) + JavaScript(입력제어)

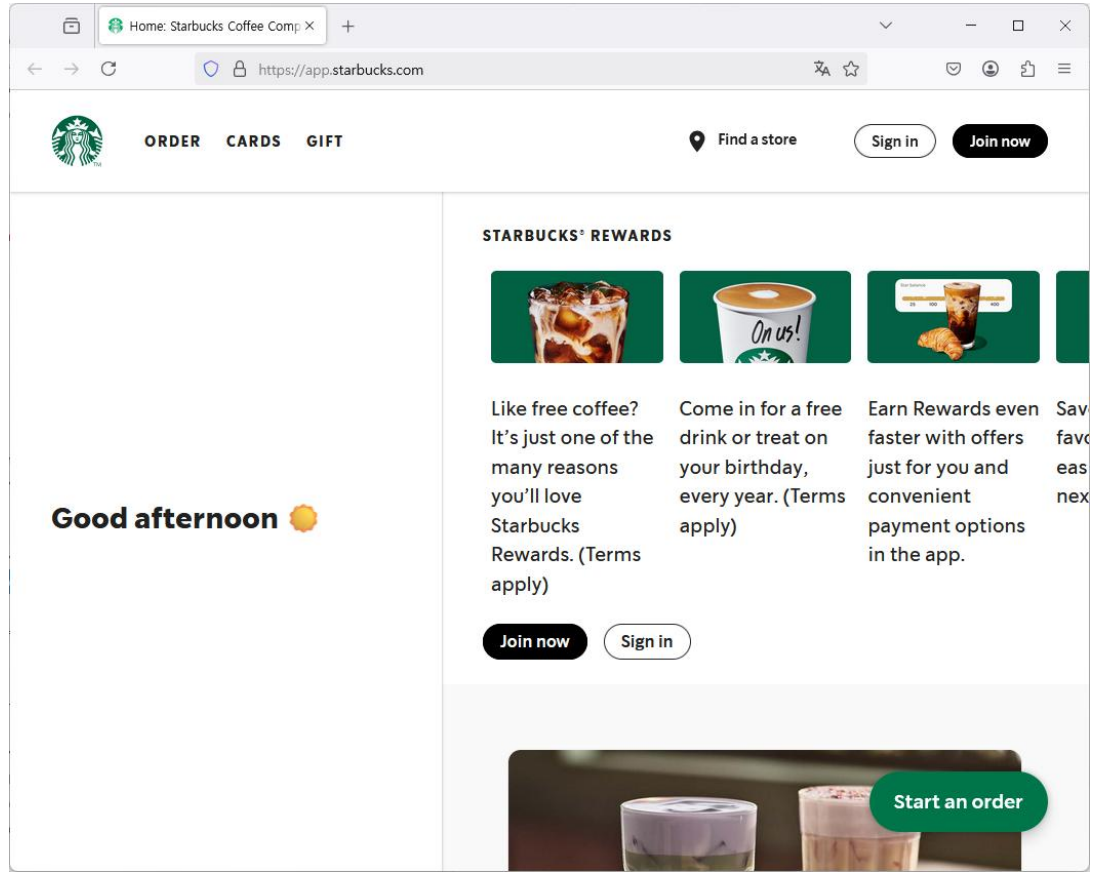




프론트엔드에서 입력 데이터 처리 방식

2. 기능별 API 요청에 따른 백엔드 처리 결과 정보를 프론트엔드에서 적극적으로 화면에 구성하여 표출
   · Dynamic HTML(화면구성) + JavaScript(입력 제어 및 API 호출제어)





프론트엔드 데이터 처리

01 프론트엔드 데이터 처리 개념

02 프론트엔드 보안 이슈

03 프론트엔드 보안 개발 방법

DOM 기반 보안 문제

정의 및 개념

DOM(Document Object Model)을 직접 조작하는 과정에서 발생하는 보안 취약점

→ 클라이언트 사이드에서 사용자 입력 또는 외부 데이터를 적절한 검증 없이 DOM에 반영하면, 공격자가 악의적 스크립트를 삽입할 수 있음

주요 특징

서버에서 처리되지 않고, 클라이언트(브라우저)에서만 발생

기존 XSS와 달리, DOM 조작 로직 내부에서 발생하는 경우가 많음

공격 유형

· DOM 기반 XSS: 문서 객체를 직접 조작하여 공격자가 임의의 스크립트를 실행시키는 공격

· 클라이언트 사이드 데이터 조작: 사용자 입력이나 URL 파라미터를 검증 없이 DOM에 삽입하는 경우

DOM 기반 보안 문제

발생 경로

- 취약한 DOM 조작 함수 사용

  `innerHTML, document.write, outerHTML` 등으로 사용자 입력을 직접 삽입

- 부적절한 입력 검증

  URL 파라미터, 해시(fragment) 값 등 외부 데이터를 제대로 필터링하지 않음

- 동적 콘텐츠 생성

  클라이언트에서 API 응답 또는 사용자 입력을 기반으로 동적으로 HTML을 생성할 때

구체적 예시

- URL 파라미터를 `location.hash`를 통해 DOM에 그대로 반영하는 경우
- 사용자 입력을 검증 없이 `element.innerHTML`에 할당하여 악성 스크립트 실행 가능

공격 시나리오

공격자가 문서의 URL을 변조해두면, 해당 페이지를 로드할 때 악성 스크립트가 실행되어 쿠키 탈취, 세션 하이재킹 등이 발생

JavaScript 실행 환경에서의 보안 문제

JavaScript 실행 환경 특성

클라이언트 브라우저에서 동적으로 코드가 실행됨

다양한 방식으로 외부 데이터를 처리하고, 동적 코드 실행(dynamically executed code)이 가능함

보안 위협의 발생 원인

- 동적 코드 실행 함수 사용

  `eval()`, `new Function()`, `setTimeout()/setInterval()`에서 문자열 인자 사용

- 글로벌 객체 노출

  `window`나 `document` 같은 전역 변수나 객체가 오염되거나, 공격자가 조작 가능한 상태로 노출

- 서드파티 스크립트 및 라이브러리

  외부 스크립트나 패키지의 취약점, 불충분한 검증 시 원치 않는 코드 실행 위험

- 프로토타입 체인

  객체의 프로토타입에 악의적 속성을 추가하여, 애플리케이션 전체에 영향을 미침

로컬 저장소의 보안 문제

로컬 저장소란?

HTML5에서 제공하는 클라이언트 측 저장소 (localStorage, sessionStorage, IndexedDB)

→ 브라우저에 데이터를 저장할 수 있는 편리한 기능

주요 활용 범위

· 사용자 선호도, 테마, 임시 캐시 데이터 저장

· 비동기 웹 애플리케이션에서 상태 관리 및 빠른 데이터 접근

장점

· 서버와의 통신 없이 빠른 데이터 읽기/쓰기 가능

· 영구적 저장 (localStorage) 또는 세션 기반 저장 (sessionStorage)

로컬 저장소의 보안 문제

XSS 공격에 의한 데이터 탈취

XSS 취약점으로 악의적 스크립트가 실행될 경우, 로컬 저장소의 모든 데이터에 접근 가능

→ 저장된 민감 정보(인증 토큰, 개인정보 등)가 공격자에게 노출

암호화 미적용

로컬 저장소는 기본적으로 평문으로 데이터를 저장하므로, 민감 정보가 그대로 노출될 위험

데이터 무결성 위협

악성 스크립트가 저장된 데이터를 수정, 삭제하거나 조작할 수 있음

클라이언트 사이드에서 쉽게 접근 및 변경이 가능

Same-Origin Policy 한계

도메인 간 격리 적용되지만, 동일 도메인 내에서는 모든 스크립트가 접근 가능

보안에 취약한 API 호출 방식

API 호출의 역할

프론트엔드에서 데이터를 주고받기 위한 핵심 통신 수단

사용자 인터랙션에 따라 동적으로 API를 호출하여 데이터를 갱신

취약한 API 호출 방식 사례

- 민감 데이터 노출
- 미암호화 통신
- 부적절한 인증/인가 처리
- 잘못된 CORS 설정

보안에 취약한 API 호출 방식

민감 데이터 노출

API 호출 시 URL의 쿼리스트링에 민감 정보를 포함할 경우, 브라우저 기록, 로그, 캐시 등에서 쉽게 탈취 가능

네트워크 통신 취약점

HTTPS 미사용, 취약한 SSL/TLS 설정 등으로 중간자 공격(MITM) 위험 증가

암호화되지 않은 채널을 통해 민감 데이터가 평문 전송되는 경우

인증 및 권한 관리 취약점

프론트엔드에서 관리되는 토큰이 안전하게 보호되지 않으면, 탈취 후 권한 상승 공격 가능

만료되지 않은 토큰 또는 재발급 절차 미흡으로 인한 부적절한 접근 제어

CORS 및 API 접근 제어 문제

잘못 구성된 CORS 정책으로 인해 외부 악의적 도메인에서도 API 호출이 가능해짐

공개 API로 오인하여 불필요한 데이터 노출 위험

프론트엔드 데이터 처리

01 프론트엔드 데이터 처리 개념

02 프론트엔드 보안 이슈

03 프론트엔드 보안 개발 방법

사용자 입력 값 검증 및 필터링

목적 및 필요성

사용자 입력이 악의적 코드(예: 스크립트, 명령어 등) 삽입 등 보안 공격의 진입점이 될 수 있음

→ 클라이언트 측에서 입력 값을 선별적으로 허용(화이트리스트)하거나,

불필요한 문자를 제거하여 XSS, Command Injection 등 다양한 취약점을 방어

검증 vs. 필터링

· 검증(Validation)

입력 값이 기대한 형식과 범위에 부합하는지 확인 (예: 이메일 형식, 숫자 범위 등)

· 필터링(Sanitization)

입력 값에서 위험한 특수문자(예: <, >, &, ;, | 등)를 제거하거나 이스케이프 처리하여 안전하게 변환

프론트엔드 역할

즉각적인 사용자 피드백 제공 및 초기 보안 방어선 역할 수행

하지만, 클라이언트 측 검증은 우회 가능하므로, 서버 측 검증이 병행되어야 함

사용자 입력 값 검증 및 필터링

HTML5 기본 검증

- `<input>` 태그의 `type`, `required`, `pattern`, `min`, `max` 등 속성을 활용해 기본적인 형식 검증
- 브라우저 내장 검증으로 사용자 경험(UX) 개선

JavaScript를 이용한 커스텀 검증

- 정규표현식을 사용하여 복잡한 형식의 유효성 검사 구현
  (예) 이메일, 전화번호, 날짜 형식 등
- 실시간 유효성 검사로 에러 메시지 및 인라인 피드백 제공

입력 값 필터링 라이브러리 활용

- DOMPurify, sanitize-html 등 서드 파티 라이브러리를 사용하여 HTML, 스크립트 태그 제거 및 안전한 값으로 변환
- 프레임워크(React, Angular, Vue 등)의 내장 기능을 활용해 XSS 공격을 사전에 차단

사용자 입력 값 검증 및 필터링

최소 허용 원칙(Whitelist) 적용

- 허용할 입력 형식을 명시적으로 정의하고, 그 외의 값은 모두 거부
- 사용자 정의 입력의 경우, 미리 정의된 안전한 패턴과 비교하여 필터링

입력 값 이스케이프 처리

- 출력 전 HTML 엔티티 인코딩을 통해 DOM 삽입 시 악의적 스크립트 실행 방지
- 서버와 클라이언트 모두에서 일관되게 적용

클라이언트와 서버 검증의 이중 방어

- 클라이언트 측은 사용자 경험을 위한 초기 검증 수단으로 활용하고,
- 반드시 서버 측에서 최종 검증 및 필터링을 수행하여 보안 취약점을 최소화

정기적인 보안 테스트 및 코드 리뷰

- 정적 분석 도구와 보안 스캐너를 활용해 입력값 처리 로직에 취약점이 없는지 점검
- 개발 및 배포 주기마다 최신 보안 모범 사례를 반영하도록 업데이트

CSP(Content Security Policy) 적용

CSP란?

허용하는 콘텐츠 출처를 명시적으로 정의하여, XSS, 데이터 인젝션 등 악의적 콘텐츠 실행을 방지하는 보안 표준

HTTP 응답 헤더(또는 HTML 메타 태그)를 통해 정책을 브라우저에 전달

CSP 적용의 필요성

· XSS 공격 완화: 스크립트 인젝션 시 공격자가 실행 가능한 스크립트 소스를 제한

· 데이터 무결성 보장: 외부 악성 리소스 로드를 방지하여, 안전한 콘텐츠만 렌더링

· 보안 강화를 위한 추가 레이어: 기존 보안 조치와 함께 적용 시 다중 방어 체계 구축



출처: https://www.securing.pl/en/why-should-you-care-about-content-security-policy/

CSP(Content Security Policy) 적용

주요 CSP 지시문

- **default-src:** 기본 콘텐츠 출처 지정 (예: 'self')
- **script-src:** 스크립트 출처 제한 (신뢰된 도메인 및 inline 스크립트 제한)
- **style-src:** 스타일시트 출처 제한
- **img-src:** 이미지 로드 출처 지정
- **connect-src:** AJAX, WebSocket, API 호출 등 네트워크 요청 출처 제한
- **report-uri / report-to:** 정책 위반 시 리포트 전송

- 구성 방법

· HTTP 헤더 설정:

(예) `Content-Security-Policy: default-src 'self'; script-src 'self' https://trusted.cdn.com; object-src 'none';`

· Meta 태그 사용 (보조 방식):

(예) `<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self'">`



요약 정리

- 프론트엔드는 SSR·CSR 방식을 통해 사용자 입력을 주고받으며, UI를 동적으로 갱신합니다.
- DOM 조작과 로컬 저장소, CORS 설정 등 다양한 보안 이슈가 존재하므로, JavaScript 실행 환경의 취약점을 인식해야 합니다.
- 입력 값 검증·필터링으로 악의적 코드 삽입을 차단하고, 서버 측 재검증을 더해 보안을 이중화 합니다.
- CSP(Content Security Policy) 를 활용해 XSS·인젝션 공격을 완화할 수 있습니다.
- 코드 리뷰와 자동화된 CI/CD 파이프라인 보안 테스트를 통해 취약점을 조기에 발견·수정해야 합니다.

참고 문헌

- [KISA] 소프트웨어 개발 보안 가이드(2021.12.29)
- [KISA] 소프트웨어 보안약점 진단가이드(2021)
- [오픈이지북스] 실무에 바로 적용하는 해킹 방어를 위한 JAVA 시큐어코딩
- [홍릉과학출판사] 소프트웨어 개발 보안 실무 - 시큐어 코딩 원리와 실제
- [secu BOOK] 웹 해킹과 시큐어코딩 탐지/수정 실습가이드
- [에이콘출판사] 보안 위협 모델링 Threat Modeling - 위협 식별과 대응을 위한 소프트웨어 설계
- [책만] 모두를 위한 소프트웨어 보안 설계와 구현 - 취약점 분석부터 위협 모델링과 완화 전략, 보안 패턴, 암호화까지 안전한 소프트웨어의 모든 것
- [위키북스] 웹 애플리케이션 보안 완벽 가이드 - 체계적으로 배우는 안전한 웹 애플리케이션 제작 기법
- [한빛미디어] 웹 애플리케이션 보안 - 정찰, 공격, 방어 세 단계로 배우는 웹 애플리케이션 보안의 모든 것
- [한빛미디어] 웹 해킹 & 보안 완벽 가이드 - 웹 애플리케이션 보안 취약점을 겨냥한 공격과 방어

감사합니다.



퀴즈

1. 다음 중 로컬 저장소(LocalStorage)에 민감 정보를 저장했을 때 발생할 수 있는 문제가 아닌 것은 ?

1) XSS 공격이 성공하면 로컬 저장소 내용이 유출될 수 있다. 2) 로컬 저장소는 기본적으로 평문으로 저장된다.

3) 도메인이 다르면 로컬 저장소를 공유할 수 있다. 4) 악성 스크립트가 로컬 저장소 내용을 임의로 변경할 수 있다.

2. SSR(Server-Side Rendering)과 CSR(Client-Side Rendering)에 대한 설명으로 옳은 것은 ?

1) SSR은 브라우저가 기본 HTML만 받고, 모든 UI를 JavaScript로 구성한다.
2) CSR은 서버에서 데이터를 주입해 완성된 HTML을 전송한다.
3) SSR은 서버에서 HTML을 완성해 응답하고, CSR은 클라이언트가 JS로 화면을 동적으로 구성한다.
4) CSR은 검색 엔진 최적화(SEO)에 매우 유리하며, SSR은 초기 로딩이 느리다.

3. 프론트엔드에서 외부 스크립트나 스타일의 로딩 출처를 제한해 XSS 같은 공격을 완화하기 위해 사용하는 보안 정책은 ? (영문 3글자)

( )



퀴즈 정답

1. **다음 중 로컬 저장소(LocalStorage)에 민감 정보를 저장했을 때 발생할 수 있는 문제가 아닌 것은 ?**

   3) 도메인이 다르면 로컬 저장소를 공유할 수 있다.

   Same-Origin Policy로 인해, 도메인이 다르면 로컬 저장소를 공유할 수 없습니다.

2. **SSR(Server-Side Rendering)과 CSR(Client-Side Rendering)에 대한 설명으로 옳은 것은 ?**

   3) SSR은 서버에서 HTML을 완성해 응답하고, CSR은 클라이언트가 JS로 화면을 동적으로 구성한다.

   SSR은 서버가 완성된 HTML을, CSR은 브라우저가 JS로 동적 화면을 구성하는 방식입니다.

3. **프론트엔드에서 외부 스크립트나 스타일의 로딩 출처를 제한해 XSS 같은 공격을 완화하기 위해 사용하는 보안 정책은 ? (영문 3글자)**

   (CSP)